Zave: Design document

Working slice · Dubai, UAE · 29 October 2026

1. Summary

Zave turns scattered bank-card promotions into a searchable, wallet-aware list. The working slice collects public Citi UAE and HSBC UAE offer pages, extracts offer details into a canonical schema, stores them in SQLite, and ranks matches by estimated value for a shopper's card and typical basket.

The prototype is deliberately honest about its evidence: it starts from reconstructed offline HTML fixtures, not a verified live crawl. It is a functional local demo, not a production or financial-advice service.

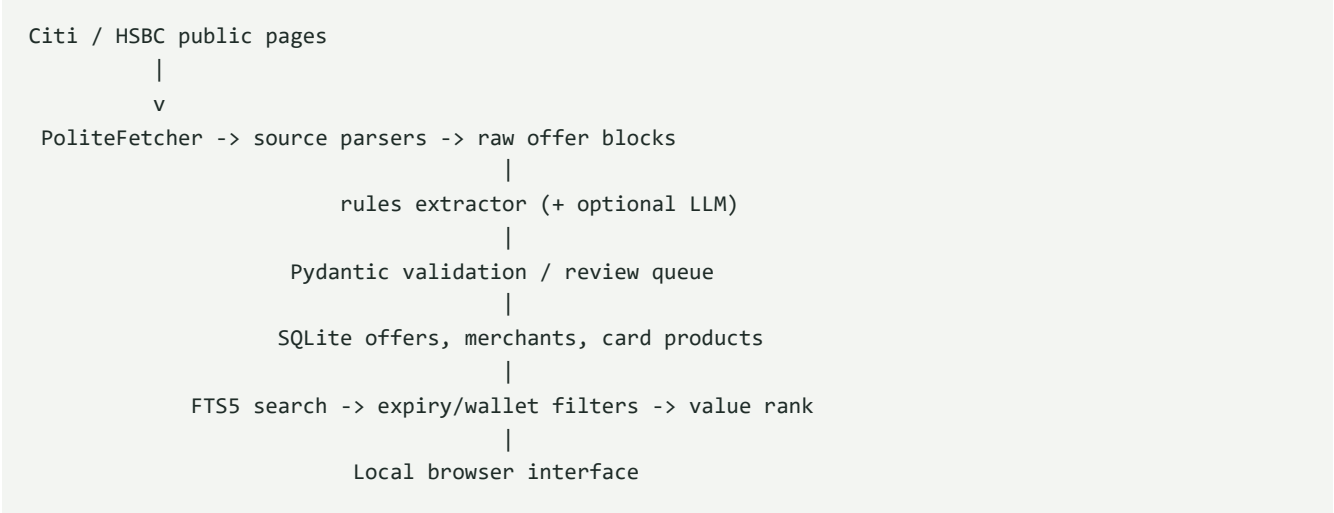
2. Problem and design goals

Card offers are split across issuer websites, use inconsistent language, and often hide important conditions in long descriptions. Shoppers have to know which card qualifies, estimate the value themselves, and check whether an offer has expired.

Design goals:

- Bring issuer offers into one normalized, searchable view.
- Filter out offers that do not match a selected issuer/card and hide expired or withdrawn offers.
- Explain the estimated AED value and surface uncertainty instead of implying a guaranteed saving.
- Keep a useful demo available offline, without requiring credentials or live access.
- Preserve source text and review uncertain or ambiguous extractions.

3. System overview



The CLI exposes collection, normalization, search, reporting, evaluation, demo, and web-preview commands. The web server is a small Python `http.server` application with JSON endpoints and a static HTML interface.

4. Data model and normalization

The canonical `Offer` record captures source and URL, merchant/title, city/category, benefit type/value/cap/minimum spend, validity, channel, promo code, exclusions, issuer/card eligibility, verification

state, source text/hash, extraction confidence, and review flags.

Benefit types include percent discount, fixed discount, BOGO, cashback, instalments, points, and non-cash perks. Controlled category/network/tier vocabularies and range/date validators reject malformed records.

Source adapters are intentionally separate from extraction:

- **Citi UAE:** parses offer blocks from the promotions page.
- **HSBC UAE:** parses offer links and sibling text from the Dubai-filtered list; detail fetching is optional.
- **Mashreq:** currently has endpoint-discovery support only; no production collector is implemented.

The rules extractor handles common percentages, caps, minimum spends, promo codes, and displayed expiry dates. An optional Groq extraction path can be compared with the rules result; disagreements on key fields reduce confidence and enter the review queue. Multi-benefit promotions are flagged rather than silently compressed into one guessed benefit.

5. Storage and lifecycle

SQLite stores normalized offers, merchant aliases, card products, a full-text search index, source-run history, application metadata, and unresolved review items. Merchant matching uses aliases, then fuzzy matching at a high threshold, and otherwise creates a new merchant.

Repeated absence from a source is treated as a lifecycle signal: an offer is withdrawn after two consecutive misses. Expiry is also a hard search filter. Raw collected blocks are cached separately from the normalized records.

The first local web run seeds fixture offers only if the database is empty. Existing data is not overwritten, and the UI labels this fixture mode as sample data.

6. Search and ranking

Search first uses SQLite FTS5 over offer and merchant text. It then removes expired/withdrawn offers, applies issuer and card-product eligibility, estimates value for the requested basket (or a category default), and sorts by a freshness/confidence-adjusted score. Expiry is only a tie-breaker, not an artificial score boost.

The estimate is not a promise of savings. Current assumptions are explicit constants in `zave/search.py`: approximate BOGO usage, cashback reliability, AED-per-point, and treatment of minimum-spend upsells. Cashback and non-cash perks are not treated as identical to immediate cash discounts. The interface advises users to check the issuer terms.

7. User experience

The browser prototype supports:

1. Browse all currently stored Dubai offers.
2. Search by merchant, offer text, or experience.
3. Filter by category and selected card products.
4. Enter a typical AED basket to personalize estimated value.
5. Open the issuer's source page to verify the terms.

The UI shows the value estimate, benefit, issuer, caveats, and expiry. It does not collect a name, account number, or payment credentials. In this local prototype, selected wallet product IDs and search inputs are sent

to the local app's API to produce results; no authentication or production privacy/security controls are implemented.

8. Reliability, compliance, and safety

The fetcher checks `robots.txt`, spaces requests, identifies itself, and treats a robots lookup failure conservatively. It does not log in or execute JavaScript to scrape offers. Before live collection, read and comply with each source's terms and robots policy; the Citi/HSBC policies have not been independently reviewed for this slice.

Offer facts can change and scraped eligibility may be incomplete. HSBC list-page card eligibility is not verified unless detail pages are fetched. Do not use the estimates as financial advice or rely on them without confirming the issuer's current terms.

9. Validation and current evidence

- `pytest -q`: **12 offline tests pass**.
- Fixture pipeline: **25 parsed offer blocks** are documented in the project notes; current tests assert 14 Citi and 11 HSBC list blocks.
- Tests cover expiry precedence, price-versus-minimum-spend parsing, wallet filtering, expiry hiding, ranking caps, withdrawal after repeated misses, and LLM disagreement handling.
- `python -m zave eval` is a small in-sample check against 13 hand-labelled examples. It is not an independent benchmark.

The HTML fixtures were reconstructed from rendered page text. DOM structure has not been validated against a live fetch. The product is therefore not represented as a live feed.

10. Known gaps and next steps

1. Run a permitted live fetch and compare extracted blocks with each issuer page; save authentic snapshots and adjust parsers where needed.
2. Review source terms/robots policies before operating scheduled collection.
3. Add a blind, stratified 150-item evaluation set and per-field quality gates before surfacing offers as verified.
4. Implement a Mashreq collector only after confirming the discovered endpoint is stable and permitted.
5. Add monitoring, freshness alerts, Arabic/PDF support, and pagination.
6. Define a deployment, authentication, privacy, and operational-security model before exposing the app publicly.

11. Run locally

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m zave web
```

Open `http://127.0.0.1:8000`. Fixture data is loaded automatically into an empty database. The current prototype is local-only; there is no public demo URL or configured GitHub remote yet.